🔐 Creds Manager

A lightweight local credentials manager built with plain HTML, CSS, JavaScript, PostgREST, and PostgreSQL. No Node.js, no frameworks — just a browser frontend talking directly to a REST API auto-generated from your database.

📐 Architecture

Layer	Tech
Frontend	`creds-manager.html` + `creds-manager.css` + `creds-manager.js`
API	PostgREST 14.12 (auto REST from PostgreSQL)
Database	PostgreSQL 18.4 (managed via pgAdmin)
Dev Server	Python HTTP Server

Service	Port
Frontend	8080
PostgREST API	3000
PostgreSQL	5432

How It Works

1. User fills in **First Name**, **Last Name**, **Email** and clicks **Add** or **Remove**
2. `creds-manager.js` sends a `fetch()` POST or DELETE request to `http://127.0.0.1:3000/creds`
3. PostgREST translates the request into a SQL `INSERT` or `DELETE`
4. PostgreSQL executes it and stores/removes the record
5. The list auto-refreshes and pgAdmin can be used to verify records directly

---

📂 File Structure

```
Creds Manager Project/
├── creds-manager.html   # Main UI
├── creds-manager.css    # Styles
└── creds-manager.js     # Fetch logic (add, remove, list)
```

🗃️ Database Setup (PostgreSQL / pgAdmin)

Open **pgAdmin → Tools → Query Tool** and run the following SQL:

1. Create the Table

sql

```sql
CREATE TABLE public.creds (
    id         BIGSERIAL PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name  TEXT NOT NULL,
    email      TEXT NOT NULL UNIQUE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

2. Create the `anon` Role & Grant Permissions

PostgREST connects using the `anon` role.

sql

```sql
-- Create the anon role (no login needed)
CREATE ROLE anon NOLOGIN;

-- Grant schema access
GRANT USAGE ON SCHEMA public TO anon;

-- Grant table permissions
GRANT SELECT, INSERT, DELETE ON TABLE public.creds TO anon;
GRANT ALL PRIVILEGES ON TABLE public.creds TO anon;

-- Grant sequence access (required for auto-increment IDs)
GRANT USAGE, SELECT ON SEQUENCE creds_id_seq TO anon;
GRANT ALL PRIVILEGES ON SEQUENCE creds_id_seq TO anon;
```

3. Verify Data

```sql
SELECT * FROM creds;
```

⚙ PostgREST Setup

PostgREST is a standalone binary that auto-generates a REST API from your PostgreSQL schema. No backend code needed.

Config File — `tutorial.conf`

```ini
db-uri = "postgres://postgres:postgres@localhost:5432/my_sample_project"
db-schema = "public"
db-anon-role = "anon"

server-host = "127.0.0.1"
server-port = 3000

server-cors-allowed-origins = "http://localhost:8080"
server-cors-allowed-headers = "Content-Type, Authorization, Accept"
server-cors-allowed-methods = "GET, POST, DELETE, OPTIONS"

log-level = "info"
```

> ⚠ The CORS settings are critical — without them the browser will block all requests from the frontend.

Start PostgREST

Open **PowerShell** and run:

```powershell
cd D:\postgrest
.\postgrest.exe tutorial.conf
```

Keep PostgREST Running (Windows Workaround)

If PostgREST exits immediately on Windows, use this loop:

```powershell
while ($true) { .\postgrest.exe tutorial.conf; Start-Sleep -Seconds 2 }
```

Verify It's Running

```cmd
netstat -ano | findstr :3000
```

Should show `LISTENING` on port 3000. Or open your browser at:

```
http://127.0.0.1:3000/creds
```

Should return `[]` or a JSON list of records.

🐍 Serving the Frontend (Python HTTP Server)

> ⚠ You **cannot** open `creds-manager.html` by double-clicking it. Browsers block `fetch()` from `file:///` pages — always serve via HTTP.

Open a **new Command Prompt** window (keep PostgREST running in its own window):

```cmd
```

```
cd "D:\Creds Manager Project"
python -m http.server 8080
```

Then open the app at:

```
http://localhost:8080
```

> 💡 You need **both terminals running simultaneously** — one for PostgREST (port 3000) and one for Python (port 8080).

✅ Startup Checklist

Follow these steps every time you want to run the app:

Step	Action
1	Open pgAdmin — make sure PostgreSQL is running
2	PowerShell → `cd D:\postgrest` → run PostgREST loop command
3	Verify PostgREST at `http://127.0.0.1:3000/creds`
4	New Command Prompt → `cd "D:\Creds Manager Project"` → `python -m http.server 8080`
5	Open `http://localhost:8080` in browser
6	Fill in fields and click **Add Creds** to test
7	Verify in pgAdmin: `SELECT * FROM creds;`

🔧 Troubleshooting

Cannot connect to PostgREST

- PostgREST is not running — check your PowerShell terminal
- Run `netstat -ano | findstr :3000` — should show `LISTENING`
- Restart using the loop command
- Make sure you're opening the page via `http://localhost:8080`, **not** `file:///`

ERR_CONNECTION_REFUSED

- PostgREST has stopped — restart it in PowerShell
- Check that port 3000 is not blocked by Windows Firewall

CORS Policy Blocked

- Check `tutorial.conf` has all three CORS lines (origins, headers, methods)
- Make sure `server-cors-allowed-origins = "http://localhost:8080"`
- Restart PostgREST after any `.conf` change

ERR_FAILED on POST

- Usually a CORS header issue — check `server-cors-allowed-headers` includes `Content-Type`
- Open **F12 → Network tab** → click the failed request → check Response Headers

Data not appearing in pgAdmin

- Run `SELECT * FROM creds;` in pgAdmin Query Tool
- Check that `anon` role has `INSERT` permission on the `creds` table
- Test with curl:

```
cmd

curl -X POST http://127.0.0.1:3000/creds -H "Content-Type: application/json" -d "{\"
```

PostgREST exits immediately on Windows

- Use the PowerShell loop command
- Check `tutorial.conf` for syntax errors
- Run `.\postgrest.exe tutorial.conf > output.txt 2>&1` and check `output.txt` for hidden errors

`anon` role does not exist

- Run in pgAdmin: `CREATE ROLE anon NOLOGIN;`
- Then re-run all `GRANT` statements from the Database Setup section

---

📋 Quick Reference Commands

powershell

```powershell
# Start PostgREST
cd D:\postgrest
.\postgrest.exe tutorial.conf

# Start PostgREST loop (Windows fix)
while ($true) { .\postgrest.exe tutorial.conf; Start-Sleep -Seconds 2 }
```

cmd

```cmd
# Start Python dev server
cd "D:\Creds Manager Project"
python -m http.server 8080

# Check PostgREST is running
netstat -ano | findstr :3000
```

sql

```sql
-- View all records (pgAdmin)
SELECT * FROM creds;

-- Clear all records (pgAdmin)
DELETE FROM creds;
```

🔗 Key URLs

URL	Purpose
`http://localhost:8080`	Frontend App
`http://127.0.0.1:3000/creds`	PostgREST API
`http://localhost/pgadmin`	pgAdmin (or use desktop app)